

Problem A. Recall

How long must a pushed copy stay?

Treat each push as a separate copy, even when several pushes use the same value. A successful query for x must refer to the most recent preceding push of x : an older copy cannot survive that push because simultaneous equal elements are forbidden.

For a push at recorded position i , let $need_i$ be the last successful query referring to this copy, or i itself if there is no such query. Compute these values in one scan. Maintain the latest push position for every value; initialize $need_i = i$ at a push, and at a successful query at position j , set $need_{lastPush[x]} = j$. The existence guarantee ensures that these required lifetimes are consistent with the negative queries.

Greedy reconstruction.

Process the recorded operations from left to right, maintaining a stack of push positions.

- For a push, append $+$ to the answer and push its recorded position.
- For either kind of query, append $?$.
- After each recorded operation at position i , repeatedly pop the stack while its top t satisfies $need_t \leq i$, appending one $-$ per pop.

An expired copy below a still-required copy must wait: only the top can be popped. When the blocker is removed, the loop removes all newly exposed expired copies as well. Keeping an active-value array or set is useful for checking the query results and ensuring that pushes do not create duplicates.

Correctness.

We show that popping an exposed copy immediately after its last required query preserves the existence of a valid completion. No future successful query needs that copy. Any later successful query for the same value belongs to a new push, before which the old copy must be removed anyway. Removing it earlier can only help a negative query or a later push.

More formally, take a valid completion and move this copy's eventual pop to the present moment. Operations on copies pushed above it in the completion can still be performed in exactly the same way. Successful queries for the removed copy do not occur before its next push, and negative queries remain valid. If it was never popped, deleting it now is also harmless. Thus there remains a valid completion after every greedy pop.

Starting from the promised valid reconstruction, apply this exchange after each recorded operation. The next recorded operation must therefore be executable with its stated result. Induction proves that the entire constructed sequence is valid. All remaining copies expire by the end of the scan; emptying the stack at that point is allowed, though not required by the problem.

Complexity.

Each pushed copy is pushed and popped once, so the stack processing takes $O(n)$ time and the output length is at most $2n$. With coordinate compression, preprocessing takes $O(n \log n)$ time and the total memory is $O(n)$. A hash table for value states gives expected $O(n)$ total time; this is the approach used by the model solution.

Problem B. Mod

We run a DP over the elements of the permuted sequence that actually decrease v when the modulo operation is applied. Let f_i be the total contribution of states in which all elements greater than i have already been considered, elements at most i have not yet been considered, and the current value is i .

Let c_i be the number of elements whose values are at most i . An operation $v \leftarrow v \bmod a_p$ decreases v exactly when a_p is the first element, in the permutation order, among the remaining candidates whose values are at most v . The contribution of a chain of such operations is the product of the corresponding

factors $1/c_v$. Therefore, for each element $a_m \leq i$, the transition is

$$f_{i \bmod a_m} \leftarrow f_{i \bmod a_m} + f_i/c_i.$$

Partition the states in $[0, V]$ into blocks of size B and compute one block at a time. For a block $[l, r]$, contributions with $a_m > r$ can be described by subtracting multiples of a_m from a source index. These contributions can be computed as correlations (difference convolutions), without counting a transition more than once. Over all blocks, this part takes $O((V^2/B) \log V)$ time. Contributions with $a_m \leq r$ can be computed directly for each state, taking $O(VB \log V)$ time in total. Choosing $B = \sqrt{V}$ gives $O(V\sqrt{V} \log V)$ time.

We can improve the bound by using different block sizes at different scales. Within $[2^j, 2^{j+1})$, the first part costs $O((V2^j/B) \log V)$ and the second part costs $O(VB)$. Choose $B = \sqrt{2^j \log V}$ for this interval. Summing over all j gives $O(V\sqrt{V} \log V)$, where V bounds n , the array values, and the initial value of v .

In an implementation, choosing B proportional to $\sqrt{r \log V}$ for a block with right endpoint r achieves the same asymptotic bound. The constant factor can be tuned to balance the two parts.

Problem C. Permutation Inversions

Turn comparisons into a graph.

Create one vertex for each position. For a constraint listing $q_1, q_2, \dots, q_k$ in increasing order of values, add edges $q_1 \rightarrow q_2, q_2 \rightarrow q_3, \dots, q_{k-1} \rightarrow q_k$. Consecutive edges imply all comparisons in that constraint, so only $k - 1$ edges are needed.

A directed cycle makes the strict comparisons impossible. Otherwise, any topological ordering $v_1, \dots, v_n$ produces a satisfying permutation by setting $p_{v_t} = t$. Notice that we output these assigned ranks by position, not the topological ordering itself.

Algorithm.

Use Kahn's topological sorting algorithm, with a min-heap containing all currently zero-indegree vertices. Repeatedly remove the smallest available position and assign it the next value $1, 2, \dots, n$. Delete its outgoing edges and insert newly available vertices into the heap. If fewer than n vertices are removed, output -1 .

The inversion-minimizing property is special to the interval constraints; the same greedy rule need not minimize inversions in an arbitrary DAG.

Key lemma.

At any stage, let x be the smallest zero-indegree vertex among the remaining vertices. Every remaining position $y < x$ is a descendant of x .

To prove this, follow predecessor edges backwards from y until reaching a source z in the remaining DAG. Such a source exists because the graph is acyclic. We have $z \geq x$. If $z = x$, the claim is immediate. Otherwise, the path from $z > x$ to $y < x$ either passes through x , or has an edge $u \rightarrow v$ whose endpoint indices lie on opposite sides of x .

This edge came from a constraint on a complete interval containing both endpoints, so that constraint also contains position x . All three positions are comparable in its prescribed total order. Since x is currently a source, neither remaining endpoint can precede x in that order. Indeed, a path from a remaining vertex to x cannot go through a previously removed vertex: topological removal never deletes a vertex while an unremoved predecessor remains. Hence x must precede both endpoints, giving a path from x to v and then to y . This proves the lemma.

Optimality.

An inversion of p corresponds exactly to an inverted pair in the topological ordering: a larger position x occurs before a smaller position y precisely when $p_x < p_y$, which makes (y, x) an inversion of p .

When the greedy algorithm removes x , any smaller position y that will appear later is still present. By the lemma, x reaches y , so every satisfying permutation must place x before y in increasing-value order.

Therefore every inversion introduced by the greedy order is forced by the constraints. Since every valid order must contain all these forced inversions, the greedy permutation has minimum possible inversion count.

Cycle detection proves that the algorithm outputs -1 exactly when no satisfying permutation exists; the lemma and the rank assignment prove correctness in every other case.

Complexity.

Let $S = \sum_{i=1}^m (r_i - l_i + 1)$. The graph has at most S edges. Constructing and processing the edges takes $O(S)$ time, and each vertex enters and leaves the heap once. The total time is $O(S + n \log n)$ and space is $O(S + n)$. Parallel edges may be retained, provided each contributes to both the indegree and its later decrement.

Problem D. Sequence

Key observation.

Construct the binary sequence s from left to right. For the current prefix, only the counts of zeros and ones matter. Denote them by (a, b) , where a counts zeros and b counts ones.

If $a > b$, complementing every bit does not change any p_i , since $[s_i \neq s_j]$ is unchanged. By symmetry, we may keep the state normalized so that $a \leq b$.

Case 1: $a = b$.

Both possible next bits have exactly a opposite bits before them, so both produce the value $p = a$. We must consume one occurrence of a from the remaining multiset. If it is absent, construction is impossible. A remaining value smaller than a also makes construction impossible, since neither count can decrease.

The two choices lead to $(a+1, b)$ and $(a, b+1)$. They give distinct sequences and symmetric continuations, so the number of choices is multiplied by two.

Case 2: $a < b$.

If an occurrence of a remains, the next bit must be 1, producing $p = a$. Choosing 0 instead would increase the smaller count to $a+1$; afterwards both counts would exceed a , so that remaining occurrence could never be produced.

If no occurrence of a remains, the next bit cannot be 1, so it must be 0, producing $p = b$. Consume an occurrence of b ; if none exists, there is no valid construction.

Thus the choice is forced whenever $a < b$. Update the counts after each step and normalize them again if necessary.

Counting the answer.

Only states with $a = b$ contribute a factor of two. Let their number be k . If all positions can be constructed while consuming exactly the given multiset, the answer is 2^k ; otherwise it is 0. Perform the arithmetic with the modulus specified in the statement.

Problem E. LCM Permutation

We first seek permutations for which the required LCM is $O(n)$.

Let $P(n)$ be a permutation a of length n such that $\text{lcm}_{1 \leq i \leq n} (a_i + a_{i+1}) = O(n)$, with $a_1 = 1$ and, when $n > 1$, $a_n = 2$. Define $Q(n)$ similarly, but using $a_i + a_{i+1} - 1$ instead. Indices are cyclic, so $a_{n+1} = a_1$.

The goal is to keep the odd parts of all adjacent sums (or sums minus one) bounded. Both $P(1)$ and $Q(1)$ are trivial. The following recursive constructions produce larger permutations. Here $[2 \mid t]$ is 1 if t is even and 0 otherwise.

1. Construct $P(2n)$ from $P(n)$ and $Q(n)$.

$$P(2n) = [2Q(n)_1 - 1, 2Q(n)_2 - 1, \dots, 2Q(n)_n - 1, 2P(n)_n, \dots, 2P(n)_1].$$

2. Construct $P(2n + 1)$ from $P(n)$ and $Q(n + 1)$.

$$P(2n + 1) = [2Q(n + 1)_1 - 1, \dots, 2Q(n + 1)_{n+1} - 1, 2P(n)_n, \dots, 2P(n)_1].$$

3. Construct $Q(2n)$ from $Q(n)$.

$$Q(2n) = [2Q(n)_1 - 1, 2Q(n)_2, \dots, 2Q(n)_n - [2 \mid (n - 1)], 2Q(n)_n - [2 \mid n], \dots, 2Q(n)_2 - 1, 2Q(n)_1].$$

4. Construct $Q(2n + 1)$ from $P(n)$.

$$Q(2n + 1) = [1, 2P(n)_1 + 1, 2P(n)_2, \dots, 2P(n)_n + [2 \mid (n - 1)], 2P(n)_n + [2 \mid n], \dots, 2P(n)_2 + 1, 2P(n)_1].$$

The properties of the smaller permutations control all adjacent pairs within each part. Since their endpoints are 1 or 2, the odd parts of the sums at the joins are bounded as well. This gives the inductive construction. Generate each required smaller permutation once and reuse it when it appears twice; the construction takes $O(n)$ time.

To obtain a sufficiently small constant, examine the joins more carefully. Their only possible odd prime factors are 3, 5, and 7. The factor 5 arises only from the exceptional endpoint $a_n = 1$ in the size-one base case.

Better base cases can be found by search for $14 \leq n < 28$. In particular, the relevant LCMs of $P(14), \dots, P(25)$ and $Q(14), \dots, Q(27)$ can all be made equal to $168 = 2^3 \cdot 3 \cdot 7$, while those of $P(26)$ and $P(27)$ can be made equal to $336 = 2^4 \cdot 3 \cdot 7$. Precompute these base permutations and apply the same recurrences for larger sizes. For $n < 14$, suitable base permutations can be found by an offline permutation search and stored in the program.

Problem F. 50 Years of Excellence

Compute the sum of the m ratings for each year.

Initialize the previous score to 0, which is the defined score immediately before the first year. Process the years from the first to the last. For each year, increase the answer if the current score is strictly less than the previous score, then store the current score as the previous score for the next comparison.

Every rating is negative, and the absolute value of a yearly score is at most $13 \cdot 1000 = 13000$. The algorithm runs in $O(nm)$ time and uses $O(1)$ additional memory.

Problem G. Toll Gates on a Tight Schedule

First remove all vertices and edges that cannot be used by a shortest route.

Let $d_1[v]$ be the distance from vertex 1 to vertex v . Let $d_n[v]$ be the distance from vertex v to vertex n . The first array is found by BFS in the original graph, and the second one by BFS in the reversed graph.

Let $D = d_1[n]$. If vertex n is unreachable, the answer is 0.

A vertex v belongs to at least one shortest route from 1 to n if and only if

$$d_1[v] + d_n[v] = D.$$

An edge $u \rightarrow v$ belongs to at least one shortest route if and only if both endpoints satisfy the condition above and

$$d_1[u] + 1 = d_1[v].$$

After keeping only these vertices and edges, we get a DAG. Its vertices are split into layers by their value of d_1 , and every edge goes from one layer to the next one.

Now compute $p[v]$, the number of paths from 1 to v in this DAG. This is done in increasing order of layers.

Let $P = p[n]$ be the total number of shortest routes. Put

$$B = \lfloor \sqrt{P} \rfloor.$$

Color a vertex black if $p[v] \leq B$, and white otherwise.

For every edge $u \rightarrow v$ in the DAG, we have $p[u] \leq p[v]$. Therefore, on any route from 1 to n , the color can change from black to white at most once.

If $P = 1$, the route is unique and can be counted directly. Otherwise, vertex 1 is black and vertex n is white, so every shortest route crosses exactly one edge from a black vertex to a white vertex.

We will count routes by this crossing edge.

Consider a crossing edge $u \rightarrow v$, where u is black and v is white.

Since u is black, the number of prefixes from 1 to u is at most B .

Since v is white, we have $p[v] > B$. Let $q[v]$ be the number of paths from v to n . Every path from 1 to v can be joined with every path from v to n , hence

$$p[v]q[v] \leq P.$$

Thus

$$q[v] < \frac{P}{B}.$$

So for every crossing edge, both the number of possible prefixes and the number of possible suffixes are $O(\sqrt{P})$.

It remains to store their sums.

For every black vertex u , store all sums of paths from 1 to u . Denote this list by $\text{pre}[u]$. The sum includes both endpoints.

For every white vertex v , store all sums of paths from v to n . Denote this list by $\text{suf}[v]$. The sum also includes both endpoints.

All tolls are nonnegative, so any partial sum greater than C can be discarded immediately.

Now take a crossing edge $u \rightarrow v$. A shortest route crossing this edge is obtained by choosing one value

$$x \in \text{pre}[u]$$

and one value

$$y \in \text{suf}[v].$$

The edge itself has no cost, and the two parts do not share vertices. Therefore, the total toll is

$$x + y.$$

We need to count pairs satisfying

$$x + y \leq C.$$

Sort one or both lists and count these pairs by binary search or two pointers. Add the result to the answer. The answer is stored in an unsigned 64-bit integer, so overflow gives the required modulo 2^{64} automatically.

It remains to justify the size bound.

Suppose the shortest path DAG has layers of sizes $s_0, s_1, \dots, s_D$. A shortest route chooses one vertex from each layer, so the number of shortest routes is at most

$$\prod_i s_i.$$

For a fixed total number of vertices, this product is maximized when the layer sizes are mostly 3. Hence

$$P = O(3^{n/3}).$$

Therefore

$$\sqrt{P} = O(3^{n/6}).$$

There are at most n relevant lists, and each relevant transition processes such lists. A safe upper bound for the running time is

$$O((n + m)n3^{n/6}).$$

The memory consumption is

$$O(n3^{n/6}),$$

if lists that are no longer needed are discarded.

Problem H. the-first-problem

First, transform the way we compute $score(S, pref(T, x))$:

Enumerate each prefix $pref(S, i)$ of S . If it occurs at positions $[p - i + 1, p]$ in T with $p \leq x$, its contribution is $x - p + 1$.

For each query, build an AC automaton. Enumerating a node u is equivalent to enumerating prefixes of the pattern strings.

Let $node_i$ be the automaton state after matching $pref(T, i)$. Inspecting the nodes inside the subtree of u in the failure tree is equivalent to inspecting all positions in T where u occurs.

Thus, to compute the answer for a fixed x , we only need to enumerate each node u and compute:

1. cnt_u : the number of patterns in this query that have u as a prefix. 2. sum_1 : the number of $node_i$ ($i \leq x$) in the subtree of u . 3. sum_2 : the sum of $(-i + 1)$ over all $node_i$ ($i \leq x$) in the subtree of u .

The contribution of u is $cnt_u(sum_1 \cdot x + sum_2)$.

Building a separate automaton for each query would require recomputing the $\{node_i\}$ sequence every time, which is too slow. Build a single automaton for all queries together.

At this point, one could binary search the answer for each query independently with online 2D range counting — the time and space complexity is suboptimal.

A better approach is parallel binary search (a.k.a. whole binary search) with 1D range counting, achieving $O(n \log^2 n)$ or $O(n \log n)$ time complexity. The reference solution (std) implements the $O(n \log^2 n)$ version.

Parallel binary search means: use divide-and-conquer to binary search the answer for all queries simultaneously. When processing the answer range $[l, r]$, maintain all queries whose answers lie in this interval, check whether each query's answer is $\leq mid$, and dispatch them into the left half $[l, mid]$ or the right half $[mid + 1, r]$ accordingly. The invariant is that upon entering $[l, r]$, all $node_i$ with $i < l$ have already been inserted into the 1D data structure.

Problem I. A Lamp of Moon

Tutorial

1. Counting the weight by slope

Define

$$f(N, p/q) = \# \left\{ (i, j) : 1 \leq i, j \leq N, \frac{j}{i} \leq \frac{p}{q} \right\}.$$

Since

$$\omega(\gcd(i, j)) + 1 = 1 + \sum_{\substack{r|i, r|j \\ r \text{ is prime}}} 1,$$

the total weight of all points whose slope is at most p/q is

$$\text{check}(p/q) = f(n, p/q) + \sum_{\substack{r \leq n \\ r \text{ is prime}}} f\left(\left\lfloor \frac{n}{r} \right\rfloor, p/q\right).$$

Group the primes by the value $v = \lfloor n/r \rfloor$. For a division block $[l, r]$, the number of primes in the block is

$$\pi(r) - \pi(l - 1).$$

A single Min25 sieve can precompute all required values of π at positions of the form $\lfloor n/x \rfloor$. Therefore one check can be written as

$$\text{check}(p/q) = f(n, p/q) + \sum_v B_v f(v, p/q),$$

where every coefficient B_v depends only on n and can be stored before processing the queries.

Please note that it's an important optimization to skip blocks whose coefficient B_v equals to zero instead of calculate their corresponding $f(v, p/q)$ with an $O(\log n)$ cost

2. Computing $f(N, p/q)$

For a fixed horizontal coordinate i , the valid vertical coordinates satisfy

$$1 \leq j \leq \min\left(N, \left\lfloor \frac{pi}{q} \right\rfloor\right).$$

Consequently,

$$f(N, p/q) = \sum_{i=1}^N \min\left(N, \left\lfloor \frac{pi}{q} \right\rfloor\right).$$

First find the last i for which the upper bound has not reached N . The remaining terms contribute N directly. The first part is a standard floor sum and can be computed with the Euclidean floor sum algorithm in $O(\log n)$ time.

3. Locating the answer with the Stern–Brocot tree

$\text{check}(x)$ is nondecreasing as the slope x increases. Maintain two neighboring fractions in the Stern–Brocot tree,

$$\frac{a}{b} < \frac{c}{d},$$

such that the entire left fraction can be taken, while the entire right fraction cannot be taken. Initially, the two boundaries are $0/1$ and $1/0$.

Check the mediant

$$\frac{a+c}{b+d}.$$

If its entire slope block can be taken, move the left boundary to the right. Otherwise, move the right boundary to the left. Only reduced fractions whose numerator and denominator are both at most n are relevant.

Moving by only one mediant at a time is too slow. For example, an answer close to $(n-1)/n$ creates a one direction chain of length $\Theta(n)$. During consecutive moves in the same direction, candidate fractions have the form

$$\frac{a+kc}{b+kd} \quad \text{or} \quad \frac{c+ka}{d+kb}.$$

Use exponential search on k , followed by binary search, to find the longest legal part of the chain. The total number of checks along all chains is $O(\log n)$. During one query, a fraction tested while jumping over a chain can become the next mediant, so previously computed **check** values should be cached.

At the end, we obtain neighboring fractions **low** and **high**. There is no other reduced fraction with numerator and denominator at most n between them, and

$$\text{check}(\text{low}) \leq L < \text{check}(\text{high}).$$

4. Why the interior of an equal slope ray is unnecessary

Let the reduced direction of **high** be (q, p) . The first point on this ray is (q, p) . Its greatest common divisor is 1, so its weight is always

$$\omega(1) + 1 = 1.$$

If $L = \text{check}(\text{low})$, the next point cannot be taken, so the answer slope is **low**. If $L > \text{check}(\text{low})$, at least 1 unit of budget remains, so the first point on the **high** ray can certainly be taken. The answer slope is therefore **high**.

The problem asks only for the slope. It is unnecessary to determine the exact multiple on the ray, and it is also unnecessary to compute $\sum_{k \leq t} (\omega(k) + 1)$.

If L is at least the total weight of all points, output the maximum slope $n/1$ directly.

5. Complexity

The time complexity of the Min25 preprocessing is

$$O\left(\frac{n^{3/4}}{\log n}\right),$$

and its space complexity is $O(\sqrt{n})$.

One **check** processes $O(\sqrt{n})$ division blocks, and each block uses an $O(\log n)$ Euclidean floor sum. Each query needs $O(\log n)$ different **check** calls. Therefore, the total time complexity is

$$O\left(\frac{n^{3/4}}{\log n} + T\sqrt{n}\log^2 n\right),$$

with space complexity $O(\sqrt{n})$.

Problem J. string

Consider the first operation as an example.

If we regard **N** as 0 and **H** as 1 (regard left side as lowbit and right side as highbit), performing the operation k times is equivalent to adding k to the original number.

We can pack every 64 bits into one machine word, similarly to the idea behind a bitset. The operation can then be performed in $O(n/w)$ time: add the carry produced by the lower word, and then continue propagating the carry.

For a walk to the right, we can reverse the binary number, perform the operation, and then reverse it back.

To exchange the meanings of **N** and **H**, we only need to take the bitwise complement of the original number, perform the operation, and then take its bitwise complement again.

The implementation may be somewhat complicated. For example, we need to reverse the bits of a binary number in $O(1)$ time.

We can preprocess the bit reversals of all 16-bit binary numbers. Suppose four 16-bit blocks a, b, c, d form a 64-bit binary number, and let a', b', c', d' denote their respective bit reversals. The reversed 64-bit number is then $d'c'b'a'$.

The time complexity is $O(nT/w)$, where $w = 64$.

Another approach is to handle the lowest $\log V$ bits by brute force. For the higher bits, each operation causes at most one range update and one point update, which can be maintained using a segment tree. This gives a time complexity of $O(n(\log n + \log V))$.

Problem K. Wolf Game

Let us call a person with value 0 a villager, and a person with value 1 a werewolf.

The main difficulty is that a statement of type **R** depends on the total number of werewolves. We will first fix this number.

Fixing the total number of werewolves

Assume that the total number of werewolves is k .

For every person i , define c_i as the number of werewolves strictly to the left of i , assuming that person i is a villager. Then the statement of a villager uniquely determines c_i : If $d_i = \text{L}$, then $c_i = b_i$. If $d_i = \text{R}$, then $c_i = k - b_i$.

Werewolves do not have to tell the truth, so their statements impose no condition.

Add two dummy villagers: position 0 before the row, with $c_0 = 0$, and position $n + 1$ after the row, with $c_{n+1} = k$.

Now consider two consecutive villagers in the extended sequence.

- If they are at positions i and $i + 1$, then there is no werewolf between them, so we need $c_i = c_{i+1}$.
- If they are at positions i and $i + 2$, then position $i + 1$ is a werewolf, so we need $c_{i+2} = c_i + 1$.

The distance between two consecutive villagers cannot be larger than 2, because there cannot be two adjacent werewolves.

Therefore, for a fixed k , the problem is equivalent to checking whether there is a path from 0 to $n + 1$ in a directed graph with edges

$$i \rightarrow i + 1 \quad \text{if } c_i = c_{i+1},$$

and

$$i \rightarrow i + 2 \quad \text{if } c_{i+2} = c_i + 1.$$

If such a path exists, all positions on the path are villagers, and all skipped positions are werewolves. This directly gives a valid construction.

DP for a fixed k

For a fixed k , let dp_i denote whether position i can be the last villager on such a path. Then

$$dp_i = (dp_{i-1} \wedge [c_i = c_{i-1}]) \vee (dp_{i-2} \wedge [c_i = c_{i-2} + 1]).$$

This is enough for an $O(n^2)$ solution by trying all values of k , but we need to do better.

How an edge changes when k changes

Each value c_i is either:

- a constant, if $d_i = L$;
- equal to k minus a constant, if $d_i = R$.

So, for any possible edge, its condition is an equality between two affine functions of k . Such an equality is either:

- true for every k ;
- false for every k ;
- true for exactly one value of k .

Thus, while sweeping $k = 0, 1, \dots, n$, every local transition changes only at some explicitly computable values of k .

Matrix representation

The recurrence only uses the previous two DP values. For every position i , we store a 2×2 boolean matrix describing the transition through position i . Boolean matrix multiplication is defined with **and** as multiplication and **or** as addition.

For position i , the matrix stores the following local possibilities: Writing its entries as $M_{11}, M_{12}, M_{21}, M_{22}$, we have $M_{11} = [c_i = c_{i-1}]$, $M_{12} = [c_{i+1} = c_{i-1} + 1]$, $M_{21} = 1$, $M_{22} = [c_i = c_{i+1}]$.

Multiplying these matrices over the whole segment gives whether a valid path through the row exists. A segment tree maintains this product while individual positions are updated.

When sweeping k , we precompute for each value of k which positions may have a changed local matrix. For every such position, we recompute its matrix and update it in the segment tree.

After all updates for the current k , the product matrix tells whether a valid path exists. If it does, we run the simple fixed- k DP once more to reconstruct one answer.

Computing the update events

Only local equalities involving positions at distance at most 2 matter. For every i , we inspect:

$$c_i = c_{i+1}$$

and

$$c_i + 1 = c_{i+2}.$$

If the directions involved are different, the equality is true for at most one value of k , so we add the corresponding affected position to the event list of that value of k . If the directions are the same, the equality is either always true or always false, and it is already handled by the initial matrices.

The implementation uses dummy positions 0 and $n + 1$ as described above, which makes the boundary cases identical to the middle of the array.

Correctness proof

Lemma 1. For a fixed total number k , an assignment is valid if and only if there is a path from 0 to $n + 1$ using only edges $i \rightarrow i + 1$ with $c_i = c_{i+1}$ and edges $i \rightarrow i + 2$ with $c_{i+2} = c_i + 1$.

Proof. Take any valid assignment and list all villagers, including dummy positions 0 and $n + 1$. Because no two werewolves are adjacent, consecutive villagers in this list have distance either 1 or 2.

If the distance is 1, no werewolf is between them, so the number of werewolves to the left is the same. If the distance is 2, exactly one werewolf is between them, so the number increases by 1. Therefore every consecutive pair gives one of the allowed edges, hence a path exists.

Conversely, given such a path, mark all positions on the path as villagers and all skipped positions as werewolves. Every gap has length 1 or 2, so no two werewolves are adjacent. The edge conditions guarantee that every villager's statement matches the actual number of werewolves on the required side. Hence the assignment is valid. $\square$

Lemma 2. For a fixed k , the DP recurrence correctly determines whether such a path exists.

Proof. The last edge entering position i can only come from $i - 1$ or $i - 2$. The first case is possible exactly when $c_i = c_{i-1}$, and the second case is possible exactly when $c_i = c_{i-2} + 1$. This is exactly the recurrence. $\square$

Lemma 3. During the sweep over k , all changes of the DP transitions are processed.

Proof. Every transition depends only on an equality between two values among c_{i-1}, c_i, c_{i+1} . Each such value is either constant or k minus a constant. Therefore each equality is either always true, always false, or true for one value of k . The event construction adds updates for precisely these single-value changes, and the always true/false cases are covered by recomputing the matrix itself. $\square$

Lemma 4. For every value of k , the segment tree product represents the same DP transitions as the fixed- k recurrence.

Proof. Each leaf matrix is recomputed from the current values of the relevant local conditions. Internal nodes store boolean products of their children, so by associativity of boolean matrix multiplication, the root stores the product of all local transitions. This product is exactly the composed DP transition over the whole row. $\square$

Theorem. The algorithm outputs -1 if and only if no valid assignment exists; otherwise, it outputs a valid assignment.

Proof. By Lemmas 1 and 2, for each fixed k a valid assignment with exactly k werewolves exists iff the fixed- k DP succeeds. By Lemmas 3 and 4, the sweep checks this condition correctly for every $k = 0, 1, \dots, n$. If no value works, no valid assignment exists. If some value works, the reconstruction runs the fixed- k DP and, by Lemma 1, produces a valid assignment. $\square$

Complexity

There are $O(n)$ events. Each event causes $O(\log n)$ work in the segment tree, and reconstruction is linear once. Therefore each test case is solved in

$$O(n \log n)$$

time and $O(n)$ memory.

Problem L. Longest Common Prefix

Let $f(i, j)$ be the largest length of a common prefix shared by some j of the first i strings, and let $L = \sum_i |s_i|$ be the total input length. For every i we must output $A_i = \sum_{j=1}^i (f(i, j) \oplus j)$.

Structural observations. Sort the strings lexicographically. If two strings share a prefix of length ℓ , then every string lying between them in sorted order shares that prefix as well, so an optimal group of j strings can always be taken to be contiguous in sorted order. Hence the common prefix of a group is the minimum of the adjacent LCPs inside it, and $f(i, j)$ equals the maximum over all windows of j consecutive sorted strings of the minimum adjacency in the window. Two consequences: $f(i, \cdot)$ is non-increasing in j (asking more strings to agree can only shorten the prefix), and it is convenient to work with the transpose $g(i, h)$, defined as the maximum number of the first i strings that share a common prefix of length h . The functions f and g are inverse non-increasing staircases: $f(i, j) \geq h$ if and only if $g(i, h) \geq j$, i.e. $f(i, j) = \max\{h : g(i, h) \geq j\}$. This transpose is the key to solving the problem incrementally, because inserting one string changes g in a very controlled way.

Brute force. For each i , sort the first i strings, compute the adjacency LCP array, and recover every $f(i, j)$ with a monotone stack (each adjacency is the minimum on a maximal window whose length caps the group sizes it can serve). Recomputing from scratch each step yields $O(n^2)$ overall, and a purely comparison-based version is $O(n^3)$. This solves the small subtasks and serves as a checker.

Main solution in $O(L \log n)$. Maintain an array whose j -th entry equals $f(i, j)$, and grow i from $i - 1$ to i by inserting s_i . Consider s_i together with the already inserted strings, and for each prefix length ℓ let $c(\ell)$ be the number of inserted strings (including s_i) that share a prefix of length at least ℓ with s_i . Since agreement on longer prefixes is more restrictive, $c(\ell)$ is non-increasing in ℓ . Its meaning is that there exist $c(\ell)$ strings with common prefix at least ℓ , so every group size $j \leq c(\ell)$ can achieve prefix length ℓ . Therefore inserting s_i amounts to the updates: for every ℓ , set the entries in the range $(c(\ell + 1), c(\ell)]$ to at least ℓ . These ranges tile $[1, c(1)]$, so one insertion issues at most $|s_i|$ range-chmax operations, $O(L)$ in total.

The prefix counts $c(\ell)$ can be obtained in two equivalent ways. With a global trie, walking s_i down the trie gives, at the node of depth ℓ , exactly the number of inserted strings passing through it, which is $c(\ell)$; this is $O(|s_i|)$ per string. Alternatively, build one suffix array over all strings joined by distinct separators; strings sharing a length- ℓ prefix with s_i occupy a contiguous range of ranks around s_i , whose endpoints are found by binary search on the LCP (RMQ over the height array), and the number of already active strings inside that range is a Fenwick-tree range sum, giving $O(|s_i| \log n)$ per string. The suffix-array route matters when an explicit trie would be too large.

It remains to maintain $\sum_j (\text{value}[j] \oplus j)$ under these operations. Two facts make this cheap. First, the maintained array is always non-increasing and the chmax values are applied in decreasing order of ℓ ; on a non-increasing array, a chmax over $[l, r]$ with value v only affects the positions whose current value is below v , and those form a suffix of $[l, r]$. So each operation is: locate the first position in $[l, r]$ with value below v , then assign that suffix to v ; a plain range-assign, handled by a single $O(\log n)$ descent. Second, the XOR-weighted sum closes in $O(1)$ on any aligned block. For a dyadic segment covering indices $[b, b + \ell)$ with $\ell = 2^k$ and b a multiple of ℓ , assigning the single value v to the whole block gives

$$\sum_{p=b}^{b+\ell-1} (p \oplus v) = \ell \cdot \left(b \oplus (v \& \neg(\ell - 1)) \right) + \frac{\ell(\ell - 1)}{2}.$$

The reason is that the low k bits of p run over $0, 1, \dots, \ell - 1$ (the low bits of b are zero), and XOR-ing with v only permutes them, so they contribute $\ell(\ell - 1)/2$ regardless of v ; meanwhile the high bits are fixed for the whole block and equal $b \oplus (v \& \neg(\ell - 1))$, contributing ℓ times that value. Thanks to this formula, whenever a segment-tree node becomes uniformly covered its XOR-sum is recomputed in $O(1)$, so each operation stays $O(\log n)$ with no extra logarithmic factor. A naive segment-tree-beats that recomputes the sum by descending would cost $O(L \log^2 n)$; the closed form removes the extra log, and a bottom-up (zkw) segment tree keeps the same $O(L \log n)$ bound with a small constant.

The simple $O(L)$ solution. Pushing the transpose one step further removes the data structure entirely. Keep, for every trie depth h , the value $g[h]$ equal to the maximum count ever seen at any node of that depth, which equals $g(i, h)$. When s_i is walked down the trie and the node of depth h reaches count cnt , there are two cases. If $\text{cnt} \leq g[h]$, then some length- h prefix was already shared by at least cnt strings, so $f[\text{cnt}] \geq h$ already holds and nothing is done. If $\text{cnt} > g[h]$, this is the first time a length- h prefix is shared by cnt strings, so raise $f[\text{cnt}]$ to h and set $g[h] = \text{cnt}$. This is correct because, for a fixed group size j , the count at a depth- h node grows through $1, 2, 3, \dots$, and at the moment it first reaches j we record $f[j] \geq h$; thus every entry receives its correct value $\max\{h : g(i, h) \geq j\}$ using point updates only, and the array stays non-increasing automatically. Keep a running total $\sum_j (f[j] \oplus j)$, updated in $O(1)$ whenever some $f[\text{cnt}]$ increases, and when advancing to query i add the new term for $j = i$. Every character of every string is visited once with $O(1)$ work, so the whole algorithm runs in $O(L)$ time and memory. This is the intended full solution.

Summary. The two crucial ingredients are the contiguity/transpose structure, which turns "choose j strings" into a monotone staircase driven by prefix counts, and the closed-form XOR-sum on aligned blocks, which lets the segment-tree solution avoid an extra logarithmic factor. The transpose viewpoint, taken to its conclusion, yields the clean $O(L)$ algorithm.

Problem M. Check In

Maintain the registration state.

Store all roster names in a set R , and keep another initially empty set C containing exactly the teams that have already signed in. Process queries in their input order. For a name s :

- If $s \notin R$, print **WRONG** and do not change either set.
- Otherwise, if $s \in C$, print **REPEAT**.
- Otherwise, print **OK** and insert s into C .

Names are case-sensitive: do not convert uppercase letters to lowercase or vice versa. In particular, repeating an invalid name still produces **WRONG**, not **REPEAT**.

Correctness.

Before each query, R is exactly the given roster and C is exactly the set of valid teams that successfully signed in during earlier queries. This holds initially because nobody has signed in. If the queried name is absent from R , it is invalid. If it belongs to both sets, it has already signed in. Otherwise it is a valid team signing in for the first time, and inserting it into C preserves the invariant. These are precisely the three required outcomes, proving correctness by induction.

Complexity.

Let L be the total number of input characters. Using balanced binary search trees, as in the model solution, the running time is $O(L \log(n+1))$: each lookup performs $O(\log(n+1))$ string comparisons, each bounded by the length of the queried name. Space is $O(L)$.

Alternatively, a hash table mapping each roster name to a signed-in flag gives expected $O(L)$ time and $O(L)$ space. A trie over the 52-letter alphabet provides a deterministic $O(L)$ -time alternative.

Problem N. Red Sequence

Prefix balances and DP.

Let $Y_i = \sum_{k=1}^i (r_k - y_k)$ and $B_i = \sum_{k=1}^i (r_k - b_k)$, with $Y_0 = B_0 = 0$. The segment $[j+1, i]$ is red exactly when $Y_j \leq Y_i$ and $B_j \leq B_i$.

Let dp_i be the maximum covered length in the first i positions. Uncovered positions can be made singleton segments. Thus the direct recurrence is

$$dp_i = \max \left(dp_{i-1}, i + \max_{0 \leq j < i, Y_j \leq Y_i, B_j \leq B_i} (dp_j - j) \right), \quad dp_0 = 0.$$

A general two-dimensional data structure is unnecessary here.

Why only the boundary matters.

There exists an optimal partition in which every non-red segment is a singleton without a red piece: split each non-red segment into singletons; any singleton containing a red piece is itself red, so this cannot decrease the answer.

Suppose the last segment is red and ends at i . If both of its balances are positive, extend it left through the preceding segments. A preceding red segment can be merged into it: both balances remain positive and no previously counted length is lost. A preceding non-red singleton has $r = 0$, so absorbing it decreases each balance by at most one. Starting with two positive integer balances, this cannot make either negative before one becomes zero. Stop as soon as a balance becomes zero, or when the segment reaches position 1.

The resulting partition is still valid and its score has not decreased. Consequently, an optimal final red segment either starts at 1, or satisfies $Y_j = Y_i$ or $B_j = B_i$. If an optimal partition ends with a non-red singleton, dp_{i-1} already covers it. These cases prove that the following restricted transitions suffice:

- $dp_i \leftarrow dp_{i-1}$;
- if $Y_i \geq 0$ and $B_i \geq 0$, set $dp_i \leftarrow i$;
- query $\max(dp_j - j)$ over previous prefixes with $Y_j = Y_i$ and $B_j \leq B_i$;
- query $\max(dp_j - j)$ over previous prefixes with $B_j = B_i$ and $Y_j \leq Y_i$.

Add i to each queried maximum and maximize dp_i with the results. Every transition constructs a valid partition, and the extension argument shows that at least one optimal partition is represented. Induction on i therefore proves correctness.

Data structures.

Group all prefix points (Y_i, B_i) by equal Y_i . In each group, sort and deduplicate the B_i coordinates and build a Fenwick tree for prefix maxima. A second family of Fenwick trees groups points by B_i and stores their Y_i coordinates. All coordinates can be prepared offline, but insert the values $dp_j - j$ only while processing prefixes in increasing order.

Initialize every Fenwick entry to negative infinity and insert prefix 0 with value 0. At step i , perform both queries before inserting prefix i , then update its position in both families with $dp_i - i$. Use an upper bound for each query so that equality is included. The answer is dp_n .

Complexity.

There are $n + 1$ prefix points, each stored once in each family. Sorting all groups and processing all queries and updates takes $O(n \log n)$ time and $O(n)$ memory. Balances lie in $[-n, n]$ and may be shifted by n for array indexing.